

GY6 Finance Management System

Technical Design Document (TDD)

1. Technical Overview

This document defines the engineering architecture and implementation design for the GY6 Finance Management System.

The system is designed as:

- modular
 - scalable
 - accounting-safe
 - transactionally secure
 - audit-friendly
-

2. Technology Stack

Frontend

- Next.js
 - React
 - TypeScript
 - Tailwind CSS
 - shadcn/ui
-

Backend

- Node.js

- API routes/services
 - Prisma ORM
-

Database

- PostgreSQL
-

Charts

- Recharts or Chart.js
-

Storage

- S3-compatible architecture
-

3. High-Level Architecture

Frontend (Next.js)



API Layer / Services



Business Logic Layer



Prisma ORM



PostgreSQL Database

4. Core System Modules

- Authentication

- Income Module
 - Expense Module
 - Accounts Module
 - Transfers Module
 - Statement Ledger
 - Dashboard Analytics
 - Export Engine
 - Audit Logging
 - Attachment Handling
-

5. Database Design

Core Tables

Financial Tables

- income_entries
- expense_entries
- statement_entries

Account Tables

- accounts
- account_categories

Operational Tables

- transfers
 - balance_adjustments
 - attachments
 - audit_logs
-

6. Ledger Architecture

The statement ledger is immutable.

Confirmed transactions create:

- ledger records
- audit logs
- balance updates

Corrections use:

- adjustments
- reversals

Never direct mutation.

7. Double-Entry Architecture

Every transaction:

- debits one account
- credits another account

Required fields:

- debit_account_id
- credit_account_id

Balance changes occur only through ledger events.

8. Transaction Processing Flow

Income Confirmation

Confirm Transaction
↓
Validate Accounts
↓
Create Statement Entry
↓
Update Balances
↓
Create Audit Log
↓
Commit Transaction

Expense Confirmation

Same transactional flow as income.

Transfer Flow

Create Transfer
↓
Validate Balances
↓
Create Statement Entry
↓
Update Accounts
↓
Audit Log
↓
Commit

9. Status Automation Engine

Automated status calculation:

if confirmed:
 No Action Required

else if due_date <= 7 days:
 Payment Needed

else:
 Payment Approaching

10. Renewal Engine

Recurring entries:

- duplicate source transaction
- preserve account linkage
- reset confirmation
- reset dates/status

Balances update only after confirmation.

11. Balance Adjustment Engine

Adjustment process:

1. compare old/new balance
 2. calculate difference
 3. create adjustment ledger entry
 4. create audit log
 5. update balance
-

12. Statement Engine

Statement system:

- immutable

- searchable
- filterable
- exportable

Supports:

- income
 - expenses
 - transfers
 - adjustments
-

13. API Architecture

Core Services

`createIncome()`

`createExpense()`

`confirmTransaction()`

`renewTransaction()`

`createTransfer()`

`createBalanceAdjustment()`

`fetchStatements()`

`fetchDashboardMetrics()`

`exportStatements()`

14. Validation Layer

Validate:

- amounts
 - linked accounts
 - dates
 - recurring references
 - transfers
 - balances
 - duplicate confirmations
-

15. Audit Logging

Audit logs required for:

- confirmations
 - transfers
 - exports
 - adjustments
 - edits
 - renewals
-

16. Security Architecture

- protected routes
 - role-based access
 - secure sessions
 - server-side validation
 - environment variable isolation
-

17. Performance Architecture

- paginated tables
- indexed queries
- optimized filtering
- lazy loading
- reusable query utilities

18. Frontend Architecture

Folder Structure

/app
/components
/services
/lib
/hooks
/types
/utils
/prisma

UI Principles

- calm UI
 - minimal clutter
 - reusable components
 - responsive layouts
 - financial readability
-

19. Testing Strategy

Critical systems requiring tests:

- ledger generation
- transfers
- balance updates
- renewals
- adjustments
- exports

20. Deployment Architecture

Recommended:

- Vercel frontend
 - PostgreSQL database
 - S3-compatible storage
-

21. Engineering Principles

Prioritize:

- correctness
- transactional safety
- accounting integrity
- scalability
- maintainability